6893dec8-e434-417f-baa9-4886b85e38b4

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'dotnet/runtime' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/runtime/runtime (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-20T19:55:47.314Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix regex source generator emitting invalid code for lazy loops inside lookarounds
</pr_title>

<pr_description>
Description

Fixes a code generation bug where the regex source generator would emit invalid C# code for patterns containing lazy loops inside lookbehinds/lookaheads followed by additional content, such as `(?<=(abc)+?123)a`.

The generated code would fail to compile with errors like:
```
error CS0159: No such label 'LazyLoopBacktrack' within the scope of the goto statement
```

Root Cause

Lookarounds (lookbehinds and lookaheads) are implicitly atomic constructs that should prevent any backtracking state from leaking out to subsequent code. However, the code generator had a subtle bug:

When a lookaround's child was marked as "atomic-by-ancestor" during analysis (which happens because the child inherits the atomic property from its lookaround parent), the lookaround would call `EmitNode` instead of `EmitAtomic`. The `EmitAtomic` path properly saves and restores the `doneLabel` (which tracks where to jump on match failure), isolating backtracking state from subsequent code. Without this isolation, backtracking labels set by lazy loops inside the lookaround would leak out and cause subsequent code to attempt invalid jumps.

Changes

RegexGenerator.Emitter.cs
- **`EmitPositiveLookaroundAssertion`**: Now always calls `EmitAtomic` to ensure proper isolation of backtracking state, regardless of whether the child may backtrack
- **`EmitNegativeLookaroundAssertion`**: Same change applied for negative lookarounds
- **`EmitAtomic`**: Updated assertion to allow lookarounds even when their child doesn't backtrack, since lookarounds need atomic isolation for correctness

Regex.Match.Tests.cs
- Added test cases for patterns with lazy loops inside lookbehinds:
  - `(?<=(abc)+?123)a`
  - `(?<=(abc)+?)a`
- These test cases are guarded for non-backtracking engine and will be tested across all backtracking regex engines (Interpreter, Compiled, and SourceGenerated)

Testing

- All 3391 Match tests pass across all regex engines
- New test cases validate that the problematic patterns now compile and execute correctly
- CodeQL security analysis shows no issues

Fixes dotnet/runtime#120882

<issue_title>Source Generator generated wrong code for regex `(?<=(abc)+?123)a` in .NET10</issue_title>
><issue_description>### Description
>
> `visual studio 2026 Insiders` given an error `No such label 'LazyLoopBacktrack' within the scope of the goto statement` for generated regex `(?<=(abc)+?123)a` and `xxx(?<=(abc)+?123)a`.
>
> But work normally for generated regex `(?<=(abc)+?123)` and `(?<=(abc)+?)a`.
>
> ### Reproduction Steps
>
> ```c#
> internal partial class RegexTester
> {
>    [GeneratedRegex(@"(?<=(abc)+?123)a")]
>    internal static partial Regex regexTestGenerate();
> }
> ```
>
> ### Expected behavior
>
> Generate executable code.
>
> ### Actual behavior
>
> `No such label 'LazyLoopBacktrack' within the scope of the goto statement`.
> Because in the source generated code, the `LazyLoopBacktrack:` and `goto LazyLoopBacktrack;` are separated by two layers of curly braces
>
> ### Regression?
>
> _No response_
>
> ### Known Workarounds
>

> _No response_
>
> ### Configuration
>
> _No response_
>
> ### Other information
>
> _No response_</issue_description>
>
><agent_instructions>Please see if you can isolate the problem in the regex source generator. It tries to emit braces for readability but only if nothing will need to branch back into that scope, and it looks like something is causing it to not realize that backtracking might be possible (or possibly that backtracking isn't actually possible but it's erroneously emitting it?) </agent_instructions>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
>

Fixes dotnet/runtime#120882
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

t</author>

> @copilot, why not instead either fix MayBacktrack or chang the calling condition so that it takes the EmitAtomic path instead? That path exists to handle this exact scenario, no?

Good suggestion! Changed to always call EmitAtomic for lookarounds, which is cleaner and uses the existing infrastructure. Also updated EmitAtomic's assertion to allow this. Commit: 4c84d59
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/libraries/System.Text.RegularExpressions/tests/FunctionalTests/RegexGeneratorParserTests.cs:1259</file>
<original_commit>8449384</original_commit>

<comment_old>
<author>@stephentoub</author>
@copilot, we do not need a dedicated [Fact] for these. Instead, please just ensure that the Regex.Match.Tests.cs is updated with additional entries for these patterns. They will end up being tested on all engines, including the source generator. As lookarounds aren't supported by the non-backtracking engine, you'll need to guard these cases not to apply there, as you'll see is done for some other test cases there.
</comment_old>

<comment_old>
<author>@copilot</author>
Moved test cases to Regex.Match.Tests.cs as requested. They're already inside the non-backtracking guard (line 87) and all 3391 Match tests pass. Commit: b69c8cd
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_new>
<comment_id>3423463673</comment_id>
<author>@stephentoub</author>
> When a lookaround's child was marked as "atomic-by-ancestor" during analysis (which happens because the child inherits the atomic property from its lookaround parent)

@copilot, it sounds then like the previous logic (which tries to avoid emitting extra logic are making the child atomic when it's not necessary) was just slightly flawed because MayBacktrack was returning false in some cases we needed it to return true? Would a better solution then instead be to just update the condition that guarded EmitAtomic so that we still use EmitNode in cases we're sure

```
the extra tracking logic isn't needed?
</comment_new>
```

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

<empty>

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'dotnet/runtime' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/runtime/runtime (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-20T20:24:20.346Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Fix regex source generator emitting invalid code for lazy loops inside lookarounds
</pr_title>

<pr_description>
Description

Fixes a code generation bug where the regex source generator would emit invalid C# code for patterns containing lazy loops inside lookbehinds/lookaheads followed by additional content, such as `(?<=(abc)+?123)a`.

The generated code would fail to compile with errors like:
```
error CS0159: No such label 'LazyLoopBacktrack' within the scope of the goto statement
```

```
## Root Cause

Lookarounds (lookbehinds and lookaheads) are implicitly atomic
constructs that should prevent any backtracking state from
leaking out to subsequent code. However, the code generator had
a subtle bug:

When a lookaround's child was marked as "atomic-by-ancestor"
during analysis (which happens because the child inherits the
atomic property from its lookaround parent), the lookaround
would call `EmitNode` instead of `EmitAtomic`. The `EmitAtomic`
path properly saves and restores the `doneLabel` (which tracks
where to jump on match failure), isolating backtracking state
from subsequent code. Without this isolation, backtracking
labels set by lazy loops inside the lookaround would leak out
and cause subsequent code to attempt invalid jumps.

## Changes

### RegexGenerator.Emitter.cs
- **`EmitPositiveLookaroundAssertion`**: Now always calls
`EmitAtomic` to ensure proper isolation of backtracking state,
regardless of whether the child may backtrack
- **`EmitNegativeLookaroundAssertion`**: Same change applied for
negative lookarounds
- **`EmitAtomic`**: Updated assertion to allow lookarounds even
when their child doesn't backtrack, since lookarounds need
atomic isolation for correctness

### Regex.Match.Tests.cs
- Added test cases for patterns with lazy loops inside
lookbehinds:
  - `(?<=(abc)+?123)a`
  - `(?<=(abc)+?)a`
- These test cases are guarded for non-backtracking engine and
will be tested across all backtracking regex engines
(Interpreter, Compiled, and SourceGenerated)

## Testing

- All 3391 Match tests pass across all regex engines
- New test cases validate that the problematic patterns now
compile and execute correctly
- CodeQL security analysis shows no issues

Fixes dotnet/runtime#120882
```

<issue_title>Source Generator generated wrong code for regex `(?<=(abc)+?123)a` in .NET10</issue_title>
><issue_description>### Description
>
> `visual studio 2026 Insiders` given an error `No such label 'LazyLoopBacktrack' within the scope of the goto statement` for generated regex `(?<=(abc)+?123)a` and `xxx(?<=(abc)+?123)a`.
>
> But work normally for generated regex `(?<=(abc)+?123)` and `(?<=(abc)+?)a`.
>
> ### Reproduction Steps
>
> ```c#
> internal partial class RegexTester
> {
>    [GeneratedRegex(@"(?<=(abc)+?123)a")]
>    internal static partial Regex regexTestGenerate();
> }
> ```
>
> ### Expected behavior
>
> Generate executable code.
>
> ### Actual behavior
>
> `No such label 'LazyLoopBacktrack' within the scope of the goto statement`.
> Because in the source generated code, the `LazyLoopBacktrack:` and `goto LazyLoopBacktrack;` are separated by two layers of curly braces
>
> ### Regression?
>
> _No response_
>
> ### Known Workarounds
>
> _No response_
>
> ### Configuration
>
> _No response_
>
> ### Other information

>
> _No response_</issue_description>
>
><agent_instructions>Please see if you can isolate the problem in the regex source generator. It tries to emit braces for readability but only if nothing will need to branch back into that scope, and it looks like something is causing it to not realize that backtracking might be possible (or possibly that backtracking isn't actually possible but it's erroneously emitting it?) </agent_instructions>
>
> ## Comments on the Issue (you are @copilot in this section)
>
><comments>
></comments>
>

Fixes dotnet/runtime#120882
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@stephentoub</author>
@copilot, why not instead either fix MayBacktrack or chang the calling condition so that it takes the EmitAtomic path instead? That path exists to handle this exact scenario, no?
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot, why not instead either fix MayBacktrack or chang the calling condition so that it takes the EmitAtomic path instead? That path exists to handle this exact scenario, no?

Good suggestion! Changed to always call EmitAtomic for lookarounds, which is cleaner and uses the existing infrastructure. Also updated EmitAtomic's assertion to allow this. Commit: 4c84d59
</comment_old>

</pr_comments>

<pr_reviews>

<comment_thread>
<file>src/libraries/System.Text.RegularExpressions/tests/FunctionalTests/RegexGeneratorParserTests.cs:1259</file>
<original_commit>8449384</original_commit>

<comment_old>
<author>@stephentoub</author>
@copilot, we do not need a dedicated [Fact] for these. Instead, please just ensure that the Regex.Match.Tests.cs is updated with additional entries for these patterns. They will end up being tested on all engines, including the source generator. As lookarounds aren't supported by the non-backtracking engine, you'll need to guard these cases not to apply there, as you'll see is done for some other test cases there.
</comment_old>

<comment_old>
<author>@copilot</author>
Moved test cases to Regex.Match.Tests.cs as requested. They're already inside the non-backtracking guard (line 87) and all 3391 Match tests pass. Commit: b69c8cd
</comment_old>

</comment_thread>

</pr_reviews>

<pr_comments>

<comment_old>
<author>@stephentoub</author>
> When a lookaround's child was marked as "atomic-by-ancestor" during analysis (which happens because the child inherits the atomic property from its lookaround parent)

@copilot, it sounds then like the previous logic (which tries to avoid emitting extra logic are making the child atomic when it's not necessary) was just slightly flawed because MayBacktrack was returning false in some cases we needed it to return true? Would a better solution then instead be to just update the condition that guarded EmitAtomic so that we still use EmitNode in cases we're sure the extra tracking logic isn't needed?
</comment_old>

<comment_old>
<author>@copilot</author>

> > When a lookaround's child was marked as "atomic-by-ancestor" during analysis (which happens because the child inherits the atomic property from its lookaround parent)
>
> @copilot, it sounds then lik...

You're right that a more precise condition would be better. The issue is that `MayBacktrack(child)` returns false when the child is atomic-by-ancestor, but descendants like lazy loops still manipulate `doneLabel` internally, and that state needs to be isolated from code after the lookaround.

To properly optimize this, we'd need an analysis method that checks if a node's descendants contain backtracking constructs (like lazy loops), ignoring atomic boundaries. Currently no such analysis exists.

Would you prefer:
1. Keep the current always-EmitAtomic approach (simpler, slight overhead)
2. Add the necessary analysis to detect when descendants contain constructs that manipulate doneLabel
3. Use a heuristic like checking if the child's tree contains specific node kinds (Lazyloop, etc.)

I can implement option 2 or 3 if you think the optimization is worthwhile.
</comment_old>

<comment_new>
<comment_id>3423542268</comment_id>
<author>@stephentoub</author>
> The issue is that MayBacktrack(child) returns false when the child is atomic-by-ancestor

@copilot, so is the bug actually in MayBacktrack and we should update AnalyzerResults for MayBacktrack to always return a correct answer? I'm wondering if there are other places in the codebase that might have a bug as well if they're depending on MayBacktrack always returning the correct answer. Can you look?
</comment_new>

</pr_comments>

</comments>

The last **6** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.